Modélisation de la propagation d'une épidémie

L'étude de la propagation des épidémies joue un rôle important dans les politiques de santé publique. Les modèles mathématiques ont permis de comprendre pourquoi il a été possible d'éradiquer la variole à la fin des années 1970 et pourquoi il est plus difficile d'éradiquer d'autres maladies comme la poliomyélite ou la rougeole. Ils ont également permis d'expliquer l'apparition d'épidémie de grippe tous les hivers. Aujourd'hui, des modèles de plus en plus complexes et puissants sont développés pour prédire la propagation d'épidémie à l'échelle planétaire telles que le SRAS, le virus H5N1 ou le virus Ebola. Ces prédictions sont utilisées par les organisations internationales pour établir des stratégies de prévention et d'intervention.

On s'intéresse ici à une méthode de simulation numérique (dite par automates cellulaires). Dans ce qui suit, on appelle grille de taille n × n une liste de n listes de longueur n, où n est un entier strictement positif. Pour mieux prendre en compte la dépendance spatiale de la contagion, il est possible de simuler la propagation d'une épidémie à l'aide d'une grille. Chaque case de la grille peut être dans un des quatre états suivants : saine, infectée, rétablie, décédée. On choisit de représenter ces quatre états par les entiers : 0 (Sain), 1 (Infecté), 2 (Rétabli) et 3 (Décédé).

L'état des cases d'une grille évolue au cours du temps selon des règles simples. On considère un modèle où l'état d'une case à l'instant t + 1 ne dépend que de son état à l'instant t et de l'état de ses huit cases voisines à l'instant t (une case du bord n'a que cinq cases voisines et trois pour une case d'un coin). Les règles de transition sont les suivantes :

– une case décédée reste décédée ;

– une case infectée devient décédée avec une probabilité p1 ou rétablie avec une probabilité (1 − p1) ; – une case rétablie reste rétablie ;

– une case saine devient infectée avec une probabilité p2 si elle a au moins une case voisine infectée et reste saine sinon.

On initialise toutes les cases dans l'état sain, sauf une case choisie au hasard dans l'état infecté.

Question 1

individu.h

```
class individu
{
  public:
  // les attributs ( data)
  char *nom,*prenom;
  int  age;
  //les méthodes
  public:
  individu(char *_nom,char *_prenomn,int _age);
  void afficherindividu();

};
```

individu.cpp

```
#include <iostream>
#include <math.h>
#include <string.h>
#include "individu.h"
using namespace std;
individu::individu(char *_nom,char *_prenom,int _age)
{       this->nom=new char[strlen(_nom)];
        this->prenom=new char[strlen(_prenom)];
        strcpy(this->nom,_nom);
        strcpy(this->prenom,_prenom);
        this->age=_age;
};
void individu::afficherindividu()
{
        cout<<this->nom<<endl;
        cout<<this->prenom<<endl;
        cout<<this->age<<endl;
};
```

main.cpp

```
#include <iostream>
#include <math.h>
#include <string.h>
#include "individu.h"
using namespace std;
main()
{
      individu I("KHALID","Mehdi",45);
```

```
        I.afficherindividu();
}
```

Proposer une classe modélisant un individu

Question 2 Implémenter les fonction membres de la classe individu

Question 3 Déclarer une matrice Population[n][n] (où n est un nombre entier non nul) représentant n^2 individus qui sont tous sains, à l'exception d'un unique individu infecté par la maladie, cet individu est choisi d'une manière au hasard.

Question 4 Ecrire une fonction int nbrSains(individu Population[][],int n) qui renvoie le nombre des individus sains.

D'après les règles de transition, pour savoir si une case saine peut devenir infectée à l'instant suivant, il faut déterminer si elle est exposée à la maladie, c'est-à-dire si elle possède au moins une case infectée dans son voisinage. Pour cela, on écrit int est_exposee(individu Population[][], int n,int i, int j) qui renvoie 1 si l'individu se trouvant dans Population[i][j] est exposé, et 0 dans le cas contraire.

Question 5 Ecrire une fonction void suivant(individu Population[][],int n, float p1, float p2) qui fait évoluer toutes les cases de la grille G à l'aide des règles de transition et renvoie une nouvelle grille correspondant à l'instant suivant. Les arguments p1 et p2 sont les probabilités qui interviennent dans les règles de transition pour les cases infectées et les cases saines. On pourra utiliser la fonction bernoulli(p) suivante qui simule une variable aléatoire de Bernoulli de paramètre p : bernoulli(p) vaut 1 avec la probabilité p et 0 avec la probabilité $(1 - p)$.

Question 6

Ecrire une fonction void simulation(individu Population[][],int n, float p1, float p2) qui réalise une simulation complète avec une grille de taille n×n pour les probabilités p1 et p2, et renvoie la liste [x0, x1, x2, x3] formée des proportions de cases dans chacun des quatre états à la fin de la simulation (une simulation s'arrête lorsque la grille n'évolue plus).

Exercice 1

un nombre Complexe est définie par sa partie réelle et sa partie imaginaire, on considère la classe suivante: Complex.h

#include <iostream>
 	
class Complexe
{
private:
float Re;
float Im;
public:
Complexe(float rr=0,float ii=0); //
void afficherComplexe();
 	
};
	#include <iostream>
	#include "Complexe.h"
	using namespace std;
	Complexe::Complexe(float rr,float ii){ this->Re=rr;this->Im=ii;};
	void Complexe::afficherComplexe(){cout<<this->Re<<"+i*"<<this->Im<<endl; };

main.cpp

```
#include <iostream>
#include "Complexe.h"
using namespace std;
int main(int argc, char** argv) {

}
```

1. Donner le résultat des appels suivants

Complexe Z(2,4); Z.afficherComplexe(); 2+i*4	Complexe Z(); Z.afficherComplexe(); 0+i*0

2 .Donner le code des fonctions membres:

- **float Complexe::Module()** permettant de calculer le module (voir exemple)

```
Complexe Z(2,1)
Z. AfficherComplexe();
2+i*1
Z.Module();
2.23606
```

- **void Complex::sommeComplexe(Complexe _Z)** permettant de calculer et de renvoyer la somme de this et _Z (voir exemple)

```
Complexe Z1(2,1);
Complexe Z2(3,2);
Z1.sommeComplexe (Z2);
Z1. AfficherComplexe();
5+i*3
Z2. AfficherComplexe();
3+i*2
```

- **void Complexe::produitComplexe(Complexe _Z)** permettant de calculer et de renvoyer le produit this* _Z

```
Complexe Z1(2,1);
Complexe Z2(3,2);
Z2. produitComplexe (Z1);
Z2. AfficherComplexe();
```

```
#include <iostream>
#include <math.h>
#include <complexe.h>
  using name space std;
Complexe:: Complexe (float _r, float _i ){This->Re=_r;
    This ->Im=_i; };
Void Complexe:
```

4+i*7
Z1. AfficherComplexe();
2+i*1

- **int Complexe::Egalité(Complexe _Z)** permettant de renvoyer **1** si this=_Z et **0** dans le cas contraire.

Complexe Z1(2,1); Complexe Z2(3,2); Z2. Egalité (Z1); affichera 0	Complexe Z1(2,1); Complexe Z2(2,1); Z2. Egalité (Z1); Affichera 1

- **Complexe Complexe ::max(Complexe _Z)** permettant de trouver et de renvoyer max(this,_Z) (on utilisera le module comme métrique)

Exercice 2:

Les nombres flottants ne permettent pas de représenter précisément les nombres rationnels. Par exemple le relationnel 1/3 ne peut être représenté que sous la forme 0.3333.. ce qui est une approximation, la solution serait de représenter les nombres rationnels par leur numérateur et leur dénominateur, sans chercher à les diviser.

```
#include <iostream>
using namespace std;
class Fraction
{
  private:
  int N,D;
  public:
  Fraction(int _n,int _d);
  void afficherFraction();

};
```

```
#include <iostream>
#include "Fraction.h"
using namespace std;

Fraction::Fraction(int _n,int _d){ this->N=_n;this->D=_d;};
void Fraction::afficherFraction(){ cout<<N<<"/"<<D<<endl;};
```

```
#include <iostream>
#include "Fraction.h"
using namespace std;
int main(int argc, char** argv) {
        Fraction R(1,2);
        R.afficherFraction();

        return 0;
}
```

1. Écrire le code de la fonction void **sommeFraction(fraction _F)** qui permet de calculer la somme this + _F

2. Écrire le code de la fonction void **ProduitFraction(Fraction _F)** qui permet de calculer le produit this* _F

3. Écrire le code de la fonction void Puissance**Fraction(int N)** qui permet de calculer $(this)^N$;
4. Écrire le code de la fonction int **SontEgales(Fraction _F)** qui renvoie 1 si this=_F et 0 dans le cas contraire.